

claude-windows / 7fe4ef45-d30a-448e-98eb-4da4fba244f1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fe4ef45-d30a-448e-98eb-4da4fba244f1\subagents\workflows\wf\_b531eaeef-39c\agent-a800384de1171dbe0.jsonl`
- SHA-256: `fe436b335467f1453b9bb544fc6c9ccfd5c6248f2ca85ba17c726ec47d6cf01a`
- Source modified: `2026-06-19T17:01:31+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_b531eaeef-39c`
- session_id: `7fe4ef45-d30a-448e-98eb-4da4fba244f1`

Transcript

1. user (2026-06-19T16:59:09.584Z)

You are an adversarial design reviewer. Be skeptical; the goal is to find real flaws BEFORE we build.

CONTEXT ? badminton/racket-sports rally detector. Pipeline: WASB shuttle DETECTOR (heatmap ? per-frame candidate list) ? single-target online TRACKER (collapses to ONE (x,y) per frame) ? velocity windower ? rallies.

MEASURED (GX010142, owner's anchor clip, on the 21-25s window, from the cached det_raw.jsonl detector candidates):

- WASB's detector emits ≥ 2 candidates on 31% of detected frames (cached, never used past the tracker).
- During the 21-25s rally: 93% of frames carry the ADJACENT-court shuttle (x-norm ~ 0.88 , what the tracker followed),
 - 83% ALSO carry a PROMINENT-court shuttle (x-norm ~ 0.56), 76% carry BOTH at once.
- A greedy nearest-neighbour reconstruction of the prominent-court candidates yields a CONTINUOUS rally arc
 - (75% window coverage, 431px vertical excursion, 8.6px median per-frame jump, ~ 2.4 s sustained)

? VISUALLY CONFIRMED

as a real foreground-court rally (player mid-stroke ? net ? second player), which the single-target tracker discarded.

PROPOSED DESIGN ("the Which problem"):

Split the tracker's two conflated jobs:

- Layer 1 "see everything": a MULTI-TARGET linker over the ALREADY-CACHED det_raw.jsonl candidates ? a SET of shuttle
 - tracks per region (top-K), each with trajectory + duration + motion-energy. Pure-Python over the cache, ZERO GPU,
 - WASB stays frozen (MIT).
- Layer 2 "pick the match's story": score each track by court-membership (the existing prominent-court polygon +
 - point_in_polygon) + rally-likeness (motion energy, vertical excursion, ≥ 2 net-crossings/Gate-A, ends on shuttle-floor-contact) and SELECT the track(s) whose story is the match's; feed those to the windower.

Claimed benefits: recall-POSITIVE (recovers the discarded prominent rally ? a G1 miss) AND precision-positive (drops the adjacent rally) ? unlike today's court_gate which is a subtractive post-filter on the single track (precision-only). Also claimed to SUBSUME the "ghost" (a stuck-pixel false-lock track has ~ 0 motion energy ? never selected). Lands default-OFF, flag-gated, measured on cached candidates + golden labels with no GPU; single-track tracker stays the default.

YOUR LENS:

Attack the COURT-ASSIGNMENT-OF-AIRBORNE-SHUTTLES weakness (the design's softest spot). The recovered prominent track rose to y~259 (near the ceiling/back wall) where, under perspective, the AIR COLUMNS of the prominent and adjacent courts OVERLAP in image space ? a high clear's (x,y) is poorly court-diagnostic at apex. Does the prominent-court polygon (a floor region, even with a pseudo-3D upward pad) reliably assign a high clear to the correct court? When does it fail? What signal disambiguates (e.g. require the track to DESCEND into the floor polygon, anchor to a striking player, full-track majority vote)? Mitigations.

Return concrete, technically-grounded risks with severity + mitigations. Mark kills_design=true ONLY for a fundamental unmitigable flaw.

2. assistant (2026-06-19T16:59:13.540Z)

I'll investigate this design's softest spot ? court assignment of airborne shuttles. Let me ground my critique in the actual code and measured data before forming risks.

3. user (2026-06-19T16:59:16.835Z)

Your configuration specifies to merge with the ref 'refs/heads/docs/court-floor-rally-end' from the remote, but no such ref was fetched.

4. user (2026-06-19T16:59:17.015Z)

```
backend\pipeline\detectors\court_gate.py
tests\test_court_gate.py
```

5. user (2026-06-19T16:59:17.035Z)

```
docs\HOW_RALLY_DETECTION_WORKS.md
```

6. assistant (2026-06-19T16:59:20.147Z)

The pull failed because this is a local branch with no remote ref ? proceeding read-only on local, which is fine for a design review. Let me read the key code and docs.

7. user (2026-06-19T16:59:21.158Z)

```
1      """Prominent-court shuttle gate ? multicourt G2 (see
2      ``docs/PROMINENT_COURT_DETECTION.md``).
3      In multicourt footage the WASB shuttle tracker follows ANY shuttle in frame, so a rally
4      on a background / adjacent court becomes a false rally on the filmed match. This gate re-
5      defines the shuttle signal ``(x, y, visible)`` ? ``(x, y, visible AND inside-the-prominent-court)``:
6      a visible shuttle point whose normalised position is outside the prominent-court polygon is marked
7      NOT visible, so the velocity windower never opens a rally there.
8
9      Pure (no GPU / cv2): point-in-polygon over the court polygon (reuses
10     ``fusion_features.point_in_polygon``). **Subtractive only** ? it can only CLEAR
11     visibility, never set it ? so on a correct polygon it suppresses out-of-court rallies without
12     manufacturing any.
13     ``polygon=None`` (or < 3 vertices, or non-positive frame dims) ? an EXACT no-op (single-
14     court / unconfigured footage), matching ``rally_gate``'s do-no-harm discipline.
15
16     The polygon is the court FLOOR in normalised 0-1 image coords (same convention as
17     ``FusionConfig.court_polygon``). ``height_pad`` is the MVP pseudo-3D play-volume (§4.3):
18     a point up to ``height_pad`` (normalised) ABOVE the floor ? i.e. higher in frame, smaller y ? is
```

```

still
18     admitted, so a high clear over the prominent court is not clipped.
19     """
20     from __future__ import annotations
21
22     from dataclasses import replace
23     from typing import Any, List, Optional, Sequence, Tuple
24
25     from backend.pipeline.detectors.fusion_features import point_in_polygon
26
27     Polygon = Sequence[Tuple[float, float]]
28
29
30     def admitted(nx: float, ny: float, polygon: Polygon, height_pad: float = 0.0) -> bool:
31         """Is the normalised point inside the court floor polygon, OR within ``height_pad``
above it?
32
33         The pseudo-3D air column: shifting the point DOWN by ``height_pad`` (toward the
floor) and
34         finding it inside means the original sits within ``height_pad`` above the floor ? a
high shuttle
35         over the prominent court, admit it."""
36         if point_in_polygon((nx, ny), polygon):
37             return True
38         return height_pad > 0.0 and point_in_polygon((nx, ny + height_pad), polygon)
39
40
41     def in_court_fraction(interval: Tuple[float, float], points: List[Any], polygon:
Polygon,
42                             fps: float, frame_width: float, frame_height: float,
43                             height_pad: float = 0.0) -> float:
44         """Fraction of a candidate rally's VISIBLE shuttle points that lie inside the
prominent court.
45
46         ``interval`` is ``(start_s, end_s)``; points carry ``.frame``. Returns 1.0 when
there are no
47         visible points in the interval (a window with no shuttle is not an out-of-court
false positive ?
48         leave that judgement to the windower)."""
49         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
50             return 1.0
51         fs, fe = interval[0] * fps, interval[1] * fps
52         n = inside = 0
53         for p in points:
54             if getattr(p, "visible", False) and fs <= p.frame <= fe:
55                 n += 1
56                 if admitted(p.x / frame_width, p.y / frame_height, polygon, height_pad):
57                     inside += 1
58         return 1.0 if n == 0 else inside / n
59
60
61     def gate_intervals_to_court(intervals: List[Tuple[float, float]], points: List[Any],
62                                polygon: Optional[Polygon], fps: float,
63                                frame_width: float, frame_height: float,
64                                min_in_court_frac: float = 0.5, height_pad: float = 0.0
65                                ) -> List[Tuple[float, float]]:
66         """Drop candidate rallies whose shuttle is MAJORITY out-of-court ? the window-level
gate.
67
68         Point-level gating (:func:`gate_points_to_court`) is leaky: a background-court rally
whose shuttle
69         occasionally drifts into the prominent court keeps a few in-court points, enough to
hold a (sparse)
70         window (measured: GX010142 rally #1 is 90% adjacent-court yet a 10% central tail

```

```

survives a hard
71         point gate). This drops an interval whose in-court point fraction is below
       ``min_in_court_frac``,
72         so a rally that is *mostly* in the adjacent court is removed wholesale. No-op
       (returns all
73         intervals) when ``polygon`` is None/degenerate. Subtractive-only."""
74         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
75             return intervals
76         return [iv for iv in intervals
77                 if in_court_fraction(iv, points, polygon, fps, frame_width, frame_height,
height_pad)
78                 >= min_in_court_frac]
79
80
81     def gate_points_to_court(points: List[Any], polygon: Optional[Polygon],
82                             frame_width: float, frame_height: float,
83                             height_pad: float = 0.0) -> List[Any]:
84         """Return ``points`` with visibility CLEARED for any visible shuttle outside the
prominent-court
85         polygon (expanded upward by ``height_pad``).
86
87         No-op (returns the input list unchanged) when ``polygon`` is None/degenerate or the
frame dims
88         are non-positive. Otherwise each gated point is a NEW ``TrajectoryPoint`` (the input
list and
89         its points are never mutated)."""
90         if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0:
91             return points
92         out: List[Any] = []
93         for p in points:
94             if getattr(p, "visible", False):
95                 nx, ny = p.x / frame_width, p.y / frame_height
96                 if not admitted(nx, ny, polygon, height_pad):
97                     p = replace(p, visible=False)
98             out.append(p)
99         return out
100

```

8. user (2026-06-19T16:59:21.671Z)

```

1     # How rally detection works (the 2-layer mental model)
2
3     A short conceptual map of how we go from raw video to rally `[start, end]` segments ?
4     and, crucially, **which layer our annotation can improve.**
5
6     ```
7     video ??? frames ??? [DETECTION] ??? shuttle (x,y,visible) per frame
8           ?                ?
9           (WASB CNN,       per-frame trajectory
10            frozen weights) ?
11                               ?
12                               velocity signal (how fast the shuttle moves)
13                               ?
14                               [SEGMENTATION] ??? the 3 knobs we tune
15                               ?
16                               ?
17                               rally windows [start, end] ??? IoU vs golden
labels
18     ```
19
20     ## Two layers ? keep them separate in your head
21
22     **1. DETECTION ? "where is the shuttle, this frame?"**
23     WASB is a CNN that, for each frame, outputs a heatmap ? the shuttle's `(x, y)` + a

```

```

24     visibility/confidence. Its weights were learned (by gradient descent, on shuttle-
coordinate
25     labels) and we use them **frozen**. We chose a *shuttle* detector on purpose: "shuttle
in
26     flight" is the signal for "a rally is happening," and it's **camera-agnostic** ? unlike
27     player-motion, which fooled the old heuristic on broadcast cameras that pan with the
players.
28     (Your videos are static tripod, so player-motion would also work ? but shuttle-motion is
the
29     robust general bet.)
30
31     **2. SEGMENTATION ? "when is a rally?"**
32     From the per-frame `(x, y)` we compute the shuttle's **velocity** (displacement between
frames).
33     Then three knobs turn that into rally intervals:
34     - **`velocity_thresh`** ? a frame is "active" only if the shuttle is moving faster than
this
35     (in flight, not lying on the floor / being picked up).
36     - **`merge_gap`** ? active frames within this many seconds get joined into one rally
(bridges
37     brief gaps: a shuttle momentarily occluded, or the beat between two hits).
38     - **`min_window_duration`** ? throw away windows shorter than this (blips / false
flickers).
39
40     Output: a list of `[start, end]` windows = the rallies.
41
42     ## Where the annotation fits (the key point)
43     Our Roora/golden labels are **rally `[start, end]` times** ? they say *when* rallies
are, not
44     *where* the shuttle is*. So they can't teach WASB's detector (that needs `(x,y)` labels).
They
45     **can** grade and tune the **segmentation layer**: we score predicted windows against
the golden
46     rallies with **temporal IoU** (how much they overlap), and pick the 3 knobs that
maximise it ?
47     **calibrate on some rallies, measure on held-out ones** so the number is honest.
48
49     ## Where sparks could land (open questions ? not commitments)
50     Things worth a shower-thought; any of these could beat 3 hand-tuned thresholds:
51     - **Richer features than raw velocity:** acceleration, trajectory *continuity* (a real
rally is a
52     smooth arc that survives occlusion), shuttle *height/arc*, or **direction reversals**
(each
53     reversal ? a shot ? ties to `shots_count`!).
54     - **A tiny learned segmenter** instead of 3 thresholds: feed the trajectory features to
a small
55     model trained on the golden rally labels (still the segmentation layer, just smarter).
56     - **Hysteresis** (two thresholds: enter-rally high, stay-in-rally low) ? robust to mid-
rally dips.
57     - **Rhythm/periodicity:** rallies have a hit cadence; dead time doesn't.
58     - **Fusion:** combine shuttle-motion (WASB) with player-motion for static cams. ?
designed in
59     [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md) (one rally-fusion layer;
the person cue is
60     ADR-007's parked #2 and `rally_gate.py`'s "Gate B" ? now built (default-OFF,
unmeasured on golden) in `fusion_hybrid.py`).
61     - **Per-video calibration as a *feature:** "watch a few labeled rallies ? auto-tune to
this
62     court/lighting ? segment the rest sharply" ? exactly your earlier idea, at the right
layer.
63
64     The whole point of per-video calibration: a handful of *your* labeled rallies nudges
these knobs
65     to *your* court/camera, and we report the IoU lift on the rallies it never saw.
66

```

9. assistant (2026-06-19T16:59:24.516Z)

Now let me read the prominent-court detection design doc and the new shuttle-floor-contact rally-END doc, plus the point_in_polygon implementation.

10. user (2026-06-19T16:59:25.162Z)

docs\PROMINENT_COURT_DETECTION.md

11. user (2026-06-19T16:59:27.004Z)

```
30: def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]]) ->
bool:
31-     """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32-     (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33-     if polygon is None or len(polygon) < 3:
34-         return False
35-     x, y = point
36-     inside = False
37-     n = len(polygon)
38-     j = n - 1
39-     for i in range(n):
40-         xi, yi = polygon[i]
41-         xj, yj = polygon[j]
42-         # Does the horizontal ray at y cross edge (i, j)?
43-         if (yi > y) != (yj > y):
44-             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45-             if x < x_cross:
46-                 inside = not inside
47-         j = i
48-     return inside
49-
50-
51: def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float, float]:
52-     """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53-     the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54-     if frame_width <= 0 or frame_height <= 0:
55-         return (0.0, 0.0)
56-     x1, _y1, x2, y2 = box
57-     return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58-
59-
60: def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float, float]:
61-     if frame_width <= 0 or frame_height <= 0:
62-         return (0.0, 0.0)
63-     x1, y1, x2, y2 = box
64-     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65-
```

12. user (2026-06-19T16:59:27.257Z)

docs\MULTI_SIGNAL_FUSION_PLAN.md

13. assistant (2026-06-19T16:59:30.167Z)

Let me read the prominent-court detection design doc (especially the §4.3 pseudo-3D height_pad section) and the new owner rally-END note.

14. user (2026-06-19T16:59:30.600Z)

```
1      # Prominent-Court Detection ? kill background-court false positives (multicourt)
2
3      > **Status:** `DRAFT` ? design + experiment **owner-gated for any default flip** .
**Owner:** Avi . **Created:** 2026-06-19 . **Last updated:** 2026-06-19
```

```

4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** §8 progress tracker is the source of truth; pointer in memory
`current-state`.
6      > **Relation to existing docs:** attacks gap **G2** ("precision ? local over-fires,
worst on multicourt") from [`RALLY_DETECTION_GAP_CLOSURE.md`](RALLY_DETECTION_GAP_CLOSURE.md);
it is the **shuttle-side** counterpart of the person-side **Gate B / court mask** in
[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md). Complements the held-shuttle
**`shuttle_player_colocation`** (G3) ? court-localization (G2) and held-shuttle (G3) are
independent precision levers.
7      > **Honesty note:** claims are tagged `[verified]` (checked vs code/measured) /
`[design]` (proposed). Every signal lands **flag-gated, default-OFF**, promoted only on measured
lift per [`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md) + [[decision-rigor-norm]].
8
9      ---
10
11     ## 0. TL;DR
12     In **multicourt** footage the camera sees several courts. The shuttle detector (WASB)
tracks *any* shuttle in frame, so a rally on a **background / adjacent court** becomes a
predicted rally on the **main match** ? a false positive. The fix: identify the **prominent
court** (the foreground court that occupies the majority of the frame, where the match being
filmed is played) and **keep only rallies whose shuttle lives inside it**. Mechanically this re-
defines the shuttle signal from `(x, y, visible)` to `(x, y, visible-AND-inside-the-prominent-
court)` **before** windowing ? so a background-court shuttle reads as "not there" and never forms
a rally. A flat 2D floor polygon is the MVP; a **pseudo-3D play-volume** (court floor + the air
column above it) is the refinement so a high clear over the prominent court isn't wrongly
rejected. Lands as a **default-OFF experiment**; the flip is gated on a Launch Report ([[launch-
report-convention]]).
13
14     ## 1. Problem & motivating example `[verified]`
15     - **Measured gap (G2):** local over-fires ~2x vs Gemini, and the over-fire is **worst on
multicourt** ? even among long (>7s) rallies, multicourt over-fires **1.58x** (background-court
games are *real* long rallies, just on the wrong court). See `RALLY_DETECTION_GAP_CLOSURE.md` §3
+ the P7 long-rally lens.
16     - **Concrete anchor ? GX010142 rally #1 (2026-06-19).** Owner observation: **"the first
rally is just a shuttle flying in the non-prominent court? while the prominent-court players
have not started."* Confirmed from the cached WASB trajectory
(`output/wasb/GX010142_1080p__wasb.csv`): rally #1 (`0.72?3.65s`, 2.94s) has a shuttle
centroid at **meanX ? 1616** on a 1920-wide frame (?84% to the right) ? spatially apart from the
central cluster where most rallies live (meanX ? 1000?1300). Several other suspicious rallies
(#3, #9, #12, #16) cluster in the same far-right band ? consistent with a **right-side adjacent
court**.
17     - **Why a 1D threshold is not enough `[verified]`: **the courts do not separate cleanly
on X or Y alone (rally Y-centroids overlap 250?620 px across courts; the far-right band mixes
with foreground play). The courts are **2D regions** under perspective ? so we need the court
*geometry*, not a coordinate cutoff. This is the owner's "pseudo-3D / court model" intuition,
validated by the data.
18
19     ## 2. Goal & non-goals
20     - **Goal:** on multicourt clips, drop rallies whose shuttle is outside the prominent
court, **without losing real prominent-court rallies** ? measured per-video, no regression on
single-court clips (where it must be a no-op or a do-no-harm).
21     - **Non-goals (v1):** multi-camera; tracking players across courts; full court-line
homography for scoring; anything biometric. Single-court footage is explicitly a **no-op** (one
court ? everything is "prominent").
22
23     ## 3. Decisions locked
24     | # | Decision | Rationale |
25     |---|-----|-----|
26     | D1 | The prominent court is a **2D region (polygon) in image space**, refined to a
**pseudo-3D play volume** | 1D thresholds don't separate courts under perspective (§1) |
27     | D2 | Gate the **shuttle trajectory** by the region **before windowing** (re-define
visibility) | Stops a background rally at the source ? no window ever forms there. Reuses
`point_in_polygon` |
28     | D3 | **Default-OFF, flag-gated; single-court = no-op** | Zero-regression rollout

```

```

([[weak-signals-retained]]); no behaviour change unless a polygon is supplied + the flag set |
29 | D4 | Reuse the existing ``court_polygon`` config + ``point_in_polygon``; do NOT invent
a new geometry stack | ``FusionConfig.court_polygon`` + ``fusion_features.point_in_polygon`` already
exist ``[verified]`` |
30 | D5 | The prominent court can be ``supplied (config) OR auto-derived``; auto-derivation
is its own measured step | Config polygon is the cheapest first experiment; auto is the product
path |
31
32 ## 4. Design
33
34 ### 4.1 The signal change (the core experiment)
35 Today the windower consumes the raw WASB trajectory: a per-frame ``(Frame, Visibility, X,
Y)``. The change is a ``pre-windowing filter``:
36
37 ```
38 for each frame f: if shuttle_visible(f) and NOT inside_prominent_court(X_f, Y_f): mark
f as NOT visible
39 ```
40
41 A background-court shuttle is then indistinguishable from "no shuttle" ? the velocity
windower never opens a rally there. This is purely ``subtractive`` on the shuttle stream (it can
only remove visibility, never add it), so on a correctly-drawn prominent court it cannot
manufacture rallies ? only suppress out-of-court ones. (Mirror of ``rally_gate``'s "do-no-harm"
discipline.)
42
43 ### 4.2 Identifying the prominent court (four routes, cheapest first)
44 1. ``Configured polygon`` (MVP, ``[design]?buildable now``): a normalized ``[[x,y],?]``
court polygon per camera/clip, exactly the existing ``FusionConfig.court_polygon``. Draw it once
from the dense shuttle/player cluster. Good enough to validate the whole concept on GX010142 +
the multicourt golden clips.
45 2. ``Player-location auto-derivation`` (``[design]``): run
``PersonDetector.detections_per_frame`` ``[verified]`` exists, take the ``largest, most-central,
most-persistent`` player cluster (the foreground players are bigger boxes lower in frame), and
fit the prominent-court polygon to where ``those`` players' feet live. "Prominent = where the
match's players are." This is why the owner asked to ``extract person features`` ? players
anchor the court.
46 3. ``Shuttle-density auto-derivation`` (``[design]``): the prominent court is the densest
sustained shuttle-activity region; cluster the visible trajectory and take the dominant mode.
Person-free fallback.
47 4. ``Court-line detection`` (``[design]``, heaviest): Hough/segmentation court lines ? the
largest court quad. Most accurate, most work; deferred.
48
49 v1 ships route 1 (config) + measures; route 2 (player-anchored) is the product upgrade.
50
51 ### 4.3 Pseudo-3D play volume (the refinement)
52 A flat floor polygon rejects a shuttle at the top of a high clear ``over the prominent
court`` (it projects above the floor quad). The pseudo-3D model treats the prominent court as a
``floor quad + a vertical air column``: accept a shuttle if its image point lies within the
floor polygon ``expanded upward`` by a height envelope (the max plausible shuttle height
projected through the camera). Practically: the floor quad plus an upward ``pad`` (perspective-
aware ? larger near the net/far baseline). Start with a generous fixed upward pad (cheap, MVP);
upgrade to a per-camera height envelope only if measurement shows real rallies being clipped.
This keeps the floor tight (rejects adjacent-court floor play) while not clipping legitimate
high shuttles.
53
54 ### 4.4 Reuse map ``[verified]``
55 | Need | Existing code |
56 |---|---|
57 | point-in-region test |
``backend/pipeline/detectors/fusion_features.py::point_in_polygon`` |
58 | normalized court polygon config |
``backend/config/models.py::FusionConfig.court_polygon`` (today: person/Gate-B only) |
59 | player boxes per frame |
``backend/pipeline/detectors/person_detector.py::PersonDetector.detections_per_frame`` |
60 | trajectory parse + windowing |

```



```

`backend/pipeline/detectors/tracknet_runner.parse_trajectory_csv`, `backend/eval/windowing.py` |
61 | per-video scoring | `backend/eval/rally_seg_eval.py` (+ the `--stratify` multicourt
split, #220) |
62
63 New code is small: a `prominent_court_gate(points, polygon, height_pad)` that filters
trajectory points, threaded into the windowing input behind a default-OFF flag.
64
65 ### 4.5 Extension ? deterministic rally-END via shuttle-floor contact (owner note,
2026-06-19) `[design]`
66 The same court geometry that localizes the prominent court also unlocks a
**deterministic rally-END signal** ? the cheapest, most certain way to know a point is over:
**the shuttle hit the floor.** This attacks **G3** (rally-END over-extension), where today's
shuttle-velocity windowing runs the window long because "shuttle still moving ? rally over."
67
68 - **Introduce two geometric primitives:**
69 - **The white court lines.** A badminton court is bounded by **white, straight lines
forming a rectangle** (with the perspective trapezoid in image space). Detect them ? Hough lines
/ line-segment detection filtered to white, long, court-consistent segments ? to recover the
**court quad** (which also **is** the prominent-court floor polygon of §4.2 route 4; one detector
serves both).
70 - **The floor plane.** The court lines define the **floor** (the plane the players
stand on). The shuttle in flight lives **above** this plane; a shuttle **at** the plane (within the
trapezoid, or just outside it) is **on the floor**.
71 - **The rally-END rule (deterministic):** a rally ends when the shuttle makes **floor
contact** ? its trajectory descends to and **stops/rests at the floor level** (a near-zero-
velocity terminus at the court-line plane), OR a shuttle that goes to the floor **outside the
prominent court** = an out-of-bounds landing (also point-over). Unlike velocity heuristics, a
shuttle resting on the floor is **unambiguous** ? the point is over.
72 - **Why it's strong + cheap:** no new model, no person stack ? it reuses the WASB
trajectory (which already gives the shuttle's descending `(x, y)` + the `Visibility` drop when
it settles) plus the court-line geometry. It is the **shuttle-side** rally-END signal,
complementary to the **player-state** rally-END (player kinematics / held-shuttle) in
`RALLY_DETECTION_GAP_CLOSURE.md` §4 Lever A ? fuse both for robustness (a shuttle can settle
off-camera; a player can pick up early).
73 - **Subtlety to measure:** distinguish a true floor landing (point over) from a shuttle
that **dips low but is played back up** (a tight net shot / low retrieve). The court-line plane
+ a brief **dwell** at floor level (the shuttle rests, doesn't re-accelerate up) is the
discriminator; tune the dwell window on golden clips.
74
75 This is tracked as **C8** below and cross-links gap-closure **G3**. It shares the court-
line detector with the prominent-court polygon (§4.2 route 4), so the two land together once
court-line detection exists.
76
77 ## 5. Experiment plan (measurement)
78 Land the gate flag-gated, then measure on the **cached trajectories** (CPU, no GPU/re-
detect):
79 1. **GX010142 sanity:** draw a prominent-court polygon; confirm **rally #1 disappears**
and the central rallies survive. The litmus the whole idea is born from.
80 2. **Multicourt golden** (`mahadevpura_2`, GX010128, Badminton_BXH_2, Boxhill_Doubles`):
? rally-count, and once those clips are golden-labelled, ? precision / ? recall vs golden via
`rally_seg_eval --stratify` (single vs multicourt). The win condition: multicourt precision up,
**recall flat** (we drop FPs, not real rallies).
81 3. **Single-court no-op:** on single-court clips with no polygon (or a full-frame
polygon) the gate is a verified no-op ? assert zero change.
82 4. **Dual-eval discipline** ([[eval-dual-set-regression]]): no per-video regression on
either eval set.
83
84 Promotion to default-ON is a **separate, owner-gated** decision with a Launch Report
(both rulers + the over-seg guard), never bundled with the experiment PR.
85
86 ## 6. Risks & limits
87 - **Polygon is camera-dependent.** A hand-drawn polygon doesn't generalize across setups
? route 2/3 (auto-derivation) is the real product path; v1 config-polygon is for **validation**.
88 - **Shuttle height** (§4.3): too-tight a floor clips high clears; the pseudo-3D pad
mitigates, measurement tunes it.

```

```

89     - **The prominent court can change** (camera re-aim mid-clip): out of scope v1 (assume
static per clip).
90     - **Not a recall fix.** This only removes out-of-court FPs; it does nothing for missed
in-court rallies (G1) ? and must be proven not to *cost* recall.
91
92     ## 7. Open questions (owner / data)
93     1. Auto-derivation anchor: **player-location** (route 2) vs **shuttle-density** (route
3) as the default once validated? *(decides: data, post-experiment)*
94     2. Height-envelope: fixed upward pad vs per-camera perspective model ? is the fixed pad
enough? *(decides: measurement)*
95     3. Do we expose the prominent court as a **one-time per-camera setup** (draw-once UI) or
always auto? *(decides: owner/product)*
96
97     ## 8. Deliverables & progress tracker ? source of truth
98     Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, off
`origin/master`, no stacking.
99
100    | ID | Deliverable | Depends on | Gated? | Status | PR |
101    |----|-----|-----|-----|-----|----|
102    | C0 | This design doc | ? | No | ? | this PR |
103    | C1 | `prominent_court_gate(points, polygon, height_pad)` + unit tests (pure, point-in-
polygon) | ? | No (default-OFF) | ? | ? |
104    | C2 | Thread the gate into the windowing input behind a default-OFF flag
(`court_polygon` + `gate_shuttle_to_court`) | C1 | No (default-OFF) | ? | ? |
105    | C3 | GX010142 litmus: rally #1 dropped, central rallies kept (measured) | C2 | No | ?
| ? |
106    | C4 | Extract person features (`detections_per_frame`) on GX010142 ? player-cluster =
prominent court; measure `shuttle_player_colocation` too | ? | No | ? | ? |
107    | C5 | Multicourt-golden measurement (? count now; ? precision/recall vs golden once
labelled) | C2, labels | ? owner labels | ? | ? |
108    | C6 | Auto-derive the polygon (route 2 player-anchored) | C4 | ? owner | ? | ? |
109    | C7 | Launch decision (default-ON) with a Launch Report | C3,C5 | ? owner | ? | ? |
110    | C8 | **Court-line / floor detection** ? (a) auto court polygon, (b) **deterministic
shuttle-floor-contact rally-END** ($4.5, targets G3) | ? | ? owner | ? | ? |
111
112    ## 9. Definition of done
113    - [ ] The gate exists, is **default-OFF**, unit-tested, and a verified no-op on single-
court / no-polygon.
114    - [ ] GX010142 rally #1 (and the far-right adjacent-court band) is dropped while the
central rallies survive `[measured]`.
115    - [ ] On the multicourt golden clips, multicourt **precision improves with recall flat**
vs golden labels (no per-video regression).
116    - [ ] A winning configuration is flipped default-ON only via a Launch Report ? flip
`Status ? DONE`, archive.
117
118    ## 10. References
119    **Code [verified]** `fusion_features.py` (`point_in_polygon`, `person_in_court`),
`person_detector.py` (`detections_per_frame`), `config/models.py`
(`FusionConfig.court_polygon`), `eval/windowing.py`, `eval/rally_seg_eval.py` (`--stratify`).
120    **Docs** [`RALLY_DETECTION_GAP_CLOSURE.md`](RALLY_DETECTION_GAP_CLOSURE.md) (G2),
[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md) (Gate B / court mask),
[`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md),
[`QUALITY_ITERATIONS.md`](QUALITY_ITERATIONS.md).
121
122    ---
123
124    ### Changelog
125    - 2026-06-19 ? Created from the owner's GX010142 rally-#1 (background-court FP)
observation + the measured multicourt over-fire (G2). Validated the 2D/pseudo-3D need from the
cached trajectory; mapped the reuse onto the existing `court_polygon` + `point_in_polygon`
machinery.
126    - 2026-06-19 ? Added $4.5 (owner note): a **deterministic rally-END signal via shuttle-
floor contact** ? detect the white court lines ? the floor plane ? a shuttle resting on the
floor (or landing out-of-court) ends the rally (targets G3). Shares the court-line detector with
the prominent-court polygon; tracked as C8.

```

15. assistant (2026-06-19T16:59:34.303Z)

I now have a strong grounding. Let me check the measured anchor data referenced ? the cached detector candidates and the recovered prominent track geometry ? to verify the perspective-overlap claim quantitatively.

16. user (2026-06-19T16:59:36.467Z)

```
total 1012
drwxr-xr-x 1 avidu 197609      0 Jun 19 22:25 .
drwxr-xr-x 1 avidu 197609      0 Jun 19 21:46 ..
-rw-r--r-- 1 avidu 197609 18149 Jun 18 13:25 long_rally_lens_n15.json
-rw-r--r-- 1 avidu 197609  3870 Jun 18 13:25 long_rally_lens_n15.txt
drwxr-xr-x 1 avidu 197609      0 Jun 19 22:22 multishuttle_142
-rw-r--r-- 1 avidu 197609  5602 Jun 19 22:23 pr245_code_review.md
-rw-r--r-- 1 avidu 197609  4028 Jun 19 22:23 pr245_fixes_plan.md
-rw-r--r-- 1 avidu 197609  8520 Jun 19 22:25 pr245_review.json
-rw-r--r-- 1 avidu 197609  5909 Jun 19 22:22 pr245_review.md
drwxr-xr-x 1 avidu 197609      0 Jun 10 15:54 rally_tal_gen0
-rw-r--r-- 1 avidu 197609 912462 Jun 16 08:50 tridet.pdf
-rw-r--r-- 1 avidu 197609  46340 Jun 16 08:50 tridet.txt
---scratch---
ABLATION_FRAMEWORK_PLAN.md
GATE_AB_NUMBERS_AND_IMPLICATIONS.md
HIGH_RISK_REMEDIATION_PLAN.md
OWNED_MODEL_PHASE0_PLAN.md
PERSONALIZATION_FEATURE_DESIGN.md
PERSONALIZATION_PIVOT_ANALYSIS.md
PR1_BODY.md
PR1_HANDOFF.md
PR2_HANDOFF.md
PR2_deferred_hardenig_status.patch
PR88_FIX.md
Rally_Detection_Quality_Report.pdf
__pycache__
_final_p1.png
_final_p4.png
_fusion_ablation_chart.png
_pov_final_p2.png
_pov_p1.png
_pov_p2.png
_rev_p1.png
_rev_p2.png
_rev_p3.png
_rev_p4.png
_rev_p5.png
_rev_p6.png
_rev_p7.png
_zoom_chart.png
_zoom_tbl1.png
ab_eval_demo.py
ab_n6_lovo.py
ablation_report.pdf
analyze_19june.py
analyze_motion.py
analyze_multishuttle_142.py
arial.ttf
at_par.py
boxhill_highlights_stage
ci_commit_msg.txt
clean_state.py
compare_19june_report.py
```

17. user (2026-06-19T16:59:37.684Z)

Found 14 files

```
docs\PROMINENT_COURT_DETECTION.md
docs\README.md
docs\RALLY_DETECTION_GAP_CLOSURE.md
docs\QUALITY_ITERATIONS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\CODE_MAP.md
docs\BACKLOG.md
docs\VIDEO_RECORDING_GUIDELINES.md
docs\archives\decisions\DECISIONS.md
docs\HOSTING_PLAN.md
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
```

18. assistant (2026-06-19T16:59:40.801Z)

The `artifacts/multishuttle\_142` directory holds the measured anchor data. Let me inspect it to ground the perspective-overlap geometry claim with the actual recovered-track numbers.

19. user (2026-06-19T16:59:41.925Z)

```
total 14832
drwxr-xr-x 1 avidu 197609      0 Jun 19 22:22 .
drwxr-xr-x 1 avidu 197609      0 Jun 19 22:25 ..
-rw-r--r-- 1 avidu 197609    2072 Jun 19 22:20 _patch_key.json
-rw-r--r-- 1 avidu 197609   790779 Jun 19 22:20 full_f1295.png
-rw-r--r-- 1 avidu 197609   820985 Jun 19 22:20 full_f1330.png
-rw-r--r-- 1 avidu 197609   804891 Jun 19 22:20 full_f1372.png
-rw-r--r-- 1 avidu 197609   815522 Jun 19 22:20 full_f1410.png
-rw-r--r-- 1 avidu 197609   815037 Jun 19 22:20 full_f1450.png
-rw-r--r-- 1 avidu 197609   814468 Jun 19 22:20 full_f1486.png
-rw-r--r-- 1 avidu 197609   821171 Jun 19 22:20 patch_01.png
-rw-r--r-- 1 avidu 197609   391114 Jun 19 22:20 patch_02.png
-rw-r--r-- 1 avidu 197609   893910 Jun 19 22:20 patch_03.png
-rw-r--r-- 1 avidu 197609   401522 Jun 19 22:20 patch_04.png
-rw-r--r-- 1 avidu 197609  1102638 Jun 19 22:20 patch_05.png
-rw-r--r-- 1 avidu 197609   463582 Jun 19 22:20 patch_06.png
-rw-r--r-- 1 avidu 197609   702892 Jun 19 22:20 patch_07.png
-rw-r--r-- 1 avidu 197609   406269 Jun 19 22:20 patch_08.png
-rw-r--r-- 1 avidu 197609   455134 Jun 19 22:20 patch_09.png
-rw-r--r-- 1 avidu 197609   759526 Jun 19 22:20 patch_10.png
-rw-r--r-- 1 avidu 197609   669678 Jun 19 22:20 patch_11.png
-rw-r--r-- 1 avidu 197609  1994937 Jun 19 22:22 tracks_overlay_crop.png
-rw-r--r-- 1 avidu 197609  1202750 Jun 19 22:22 tracks_overlay_full.png
```

20. user (2026-06-19T16:59:42.105Z)

```
1      """Read-only probe: does WASB's DETECTOR see a 2nd (prominent-court) shuttle during
2      GX010142's 21-25s 'adjacent-court' rally that the single-target TRACKER discarded?
3
4      Grounds the multi-shuttle ('Which shuttle') design question. Pure-Python, no GPU/torch.
5      Reads the already-cached raw detector candidates (det_raw.jsonl) + the tracked CSV.
6      """
7      import json
8      import math
9      import os
10     from collections import defaultdict, Counter
11
12     CACHE = "output/wasb/GX010142_1080p__c4cfa94a792f_wasb_wasbcache/det_raw.jsonl"
13     CSV = "output/wasb/GX010142_1080p__c4cfa94a792f_wasb.csv"
```

```

14     FPS = 60000 / 1001
15     W, H = 1920.0, 1080.0
16
17     # --- the owner's anchor window (rally 21-25; current-state cites 21.30-25.09) ---
18     T0, T1 = 21.0, 25.2
19     F0, F1 = int(T0 * FPS), int(T1 * FPS)
20
21     # crude x-norm buckets (perspective ignored ? first-cut, see doc §1):
22     #   prominent (central) court vs right-side adjacent court.
23     PROM_LO, PROM_HI = 0.28, 0.74
24     ADJ_LO = 0.78
25
26
27     def round_xy(x, y, q=8.0):
28         return (round(x / q) * q, round(y / q) * q)
29
30
31     def load_det_by_fid(path):
32         """frame_id -> set of unique (x,y,score) candidates (dedup the 3x window
33         repetition)."""
34         det = defaultdict(dict) # fid -> {rounded_xy: (x,y,score)}
35         with open(path) as f:
36             for line in f:
37                 line = line.strip()
38                 if not line:
39                     continue
40                 obj = json.loads(line)
41                 for fid, cand in obj["f"]:
42                     for c in cand:
43                         x, y, score, scale = c
44                         key = round_xy(x, y)
45                         # keep the highest-score instance of a near-duplicate blob
46                         prev = det[fid].get(key)
47                         if prev is None or score > prev[2]:
48                             det[fid][key] = (x, y, score)
49         return {fid: list(d.values()) for fid, d in det.items()}
50
51     def load_tracked(path):
52         tr = {}
53         with open(path) as f:
54             next(f)
55             for line in f:
56                 fr, vis, x, y = line.strip().split(",")
57                 fr = int(fr)
58                 vis = int(vis)
59                 try:
60                     x = float(x); y = float(y)
61                 except ValueError:
62                     x = y = float("-inf")
63                 tr[fr] = (vis, x, y)
64         return tr
65
66
67     def main():
68         if not os.path.exists(CACHE):
69             raise SystemExit(f"missing {CACHE}")
70         det = load_det_by_fid(CACHE)
71         tr = load_tracked(CSV)
72
73         # ----- GLOBAL: how often does the detector see >1 shuttle candidate?
74         -----
75         mult = Counter()
76         total_cand_frames = 0
77         for fid, cand in det.items():

```

```

77         if cand:
78             total_cand_frames += 1
79             mult[min(len(cand), 5)] += 1
80     print("=== GLOBAL detector-candidate multiplicity (unique blobs/frame, dedup@8px)
===")
81     print(f"frames with >=1 candidate: {total_cand_frames} of {len(det)} cached frames")
82     for k in sorted(mult):
83         label = f"{k}+" if k == 5 else str(k)
84         print(f" {label} candidate(s): {mult[k]} frames
({100*mult[k]/total_cand_frames:.1f}%)")
85         multi = sum(v for k, v in mult.items() if k >= 2)
86         print(f" >=2 candidates (multi-shuttle frames): {multi}
({100*multi/total_cand_frames:.1f}% of detected frames)")
87
88     # ----- GHOST: most frequent fixed-pixel candidate across the whole clip
-----
89     pix = Counter()
90     for fid, cand in det.items():
91         for (x, y, s) in cand:
92             pix[round_xy(x, y, q=4.0)] += 1
93     print("\n=== GHOST scan: top fixed-pixel candidate locations (dedup@4px) ===")
94     for (px, py), n in pix.most_common(6):
95         print(f" ({px:.0f},{py:.0f}) x-norm={px/W:.3f} appears in {n} frames")
96
97     # ----- THE WINDOW: prominent vs adjacent candidates during 21-25s -----
98     print(f"\n=== WINDOW {T0:.1f}-{T1:.1f}s (frames {F0}-{F1}) ===")
99     frames = range(F0, F1 + 1)
100     n_frames = 0
101     n_adj = n_prom = n_both = 0
102     tracker_xnorms = []
103     prom_xnorms = []
104     adj_xnorms = []
105     sample = []
106     for fid in frames:
107         cand = det.get(fid, [])
108         if not cand:
109             continue
110         n_frames += 1
111         xs = [c[0] / W for c in cand]
112         has_adj = any(xn >= ADJ_LO for xn in xs)
113         has_prom = any(PROM_LO <= xn <= PROM_HI for xn in xs)
114         n_adj += has_adj
115         n_prom += has_prom
116         n_both += (has_adj and has_prom)
117         prom_xnorms += [xn for xn in xs if PROM_LO <= xn <= PROM_HI]
118         adj_xnorms += [xn for xn in xs if xn >= ADJ_LO]
119         v, tx, ty = tr.get(fid, (0, float("-inf"), float("-inf")))
120         if v and math.isfinite(tx):
121             tracker_xnorms.append(tx / W)
122         if len(sample) < 12 and len(cand) >= 2:
123             sample.append((fid, sorted(round(x / W, 3) for x, y, s in cand)))
124
125     def stats(name, arr):
126         if not arr:
127             print(f" {name}: (none)")
128             return
129         arr = sorted(arr)
130         mean = sum(arr) / len(arr)
131         print(f" {name}: n={len(arr)} mean x-norm={mean:.3f} "
132               f"min={arr[0]:.3f} median={arr[len(arr)//2]:.3f} max={arr[-1]:.3f}")
133
134     print(f"frames with >=1 candidate in window: {n_frames}")
135     print(f" frames with an ADJACENT-court candidate (x-norm>={ADJ_LO}): {n_adj} "
136           f"({100*n_adj/max(1,n_frames):.0f}%)")
137     print(f" frames with a PROMINENT-court candidate (x-norm {PROM_LO}-{PROM_HI}):

```

```

{n_prom} "
138         f"({100*n_prom/max(1,n_frames):.0f}%)")
139     print(f"   frames with BOTH simultaneously: {n_both}
({100*n_both/max(1,n_frames):.0f}%)")
140     stats("tracker chose (visible)", tracker_xnorms)
141     stats("PROMINENT-court candidates", prom_xnorms)
142     stats("ADJACENT-court candidates", adj_xnorms)
143     print(" sample multi-candidate frames (fid -> sorted x-norms):")
144     for fid, xs in sample:
145         print(f"       {fid}: {xs}")
146
147
148     if __name__ == "__main__":
149         main()
150

```

21. assistant (2026-06-19T16:59:46.129Z)

Let me look at the patch key (the recovered prominent track's vertical excursion / y-coordinate detail, which is the crux of the perspective-overlap attack) and the overlay images.

22. user (2026-06-19T16:59:47.553Z)

```

1      {
2      "patch_01.png": {
3          "moment": 1295,
4          "frames": [
5              1294,
6              1295,
7              1296
8          ],
9          "loc": "prom",
10         "cx": 1180.5,
11         "cy": 461.3,
12         "x_norm": 0.615
13     },
14     "patch_02.png": {
15         "moment": 1295,
16         "frames": [
17             1294,
18             1295,
19             1296
20         ],
21         "loc": "adj",
22         "cx": 1743.8,
23         "cy": 115.3,
24         "x_norm": 0.908
25     },
26     "patch_03.png": {
27         "moment": 1330,
28         "frames": [
29             1329,
30             1330,
31             1331
32         ],
33         "loc": "prom",
34         "cx": 904.2,
35         "cy": 261.6,
36         "x_norm": 0.471
37     },
38     "patch_04.png": {
39         "moment": 1330,
40         "frames": [
41             1329,
42             1330,

```

```
43         1331
44     ],
45     "loc": "adj",
46     "cx": 1672.2,
47     "cy": 207.0,
48     "x_norm": 0.871
49 },
50 "patch_05.png": {
51     "moment": 1372,
52     "frames": [
53         1371,
54         1372,
55         1373
56     ],
57     "loc": "prom",
58     "cx": 873.3,
59     "cy": 394.0,
60     "x_norm": 0.455
61 },
62 "patch_06.png": {
63     "moment": 1372,
64     "frames": [
65         1371,
66         1372,
67         1373
68     ],
69     "loc": "adj",
70     "cx": 1695.2,
71     "cy": 414.8,
72     "x_norm": 0.883
73 },
74 "patch_07.png": {
75     "moment": 1410,
76     "frames": [
77         1409,
78         1410,
79         1411
80     ],
81     "loc": "prom",
82     "cx": 1284.0,
83     "cy": 504.5,
84     "x_norm": 0.669
85 },
86 "patch_08.png": {
87     "moment": 1410,
88     "frames": [
89         1409,
90         1410,
91         1411
92     ],
93     "loc": "adj",
94     "cx": 1786.0,
95     "cy": 171.2,
96     "x_norm": 0.93
97 },
98 "patch_09.png": {
99     "moment": 1450,
100     "frames": [
101         1449,
102         1450,
103         1451
104     ],
105     "loc": "adj",
106     "cx": 1664.6,
107     "cy": 142.6,
```



```

108     "x_norm": 0.867
109 },
110 "patch_10.png": {
111     "moment": 1486,
112     "frames": [
113         1485,
114         1486,
115         1487
116     ],
117     "loc": "prom",
118     "cx": 1038.8,
119     "cy": 636.3,
120     "x_norm": 0.541
121 },
122 "patch_11.png": {
123     "moment": 1486,
124     "frames": [
125         1485,
126         1486,
127         1487
128     ],
129     "loc": "adj",
130     "cx": 1574.7,
131     "cy": 215.7,
132     "x_norm": 0.82
133 }
134 }

```

23. assistant (2026-06-19T16:59:55.631Z)

This overlay image is the smoking gun for my lens. I can see directly:

- Green = the discarded prominent-court track. It rises to a high clear apex near the top of the frame (near ceiling).
- Yellow = the tracker output (adjacent court, right side).

Critically, looking at the patch key data: at moment 1330, the prominent shuttle is at $cy=261.6$ (x_norm 0.471) while the adjacent is at $cy=207$ (x_norm 0.871). At moment 1410, the prominent rose to a clear but the adjacent shuttle is at $cy=171$? high in the frame. The air columns visibly overlap in the upper third of the frame.

Let me examine the overlay crop and one full frame to confirm where the prominent apex lands relative to the adjacent court's floor region, since that's the exact failure mode (a prominent high clear landing in y-space where the adjacent court's air column / even floor-pad lives).

24. assistant (2026-06-19T17:00:08.736Z)

This crop makes the attack vivid and confirms the geometry. Key observations directly visible:

1. **The green (prominent) track's high-clear apex** rises to roughly the same vertical band (image y) as where the **yellow (adjacent) track lives** ? the adjacent court's entire play, including its near-floor returns, sits in the upper-right where the green apex also reaches. The two air columns overlap in image-y.

2. **Decisive failure illustration:** the green prominent-court apex (left arc, peaking ~y 60-90 in this crop) is **higher in the frame** (smaller y) than much of the yellow adjacent-court play. A height_pad that pads the prominent floor polygon **upward** to admit this apex would extend the prominent admission region up into a y-band where adjacent-court shuttles also appear. And conversely, the adjacent court's own floor + air column reaches high. The polygon is a floor region; padding it up to catch clears necessarily bleeds into the neighboring court's image-space air column.

3. The green track's apex is also far LEFT (x_norm ~0.1-0.2 at apex per the arc) ? meaning a high clear's x drifts too, not just y.

I have everything I need to ground the risks concretely. Let me verify one more thing ? whether

`court\_polygon` is a single static polygon and whether any per-frame descent/track-direction signal exists today.

25. user (2026-06-19T17:00:10.177Z)

```
88- shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
89- true ? the person-cue features, but it does NOT gate the served handoff unless a
90- threshold is raised (`min_crossings`/`min_players` default 0 = OFF). The court mask
91- is optional: `court_polygon=None` ? Gate B counts every detection (player-count-only);
92- supplied ? off-court persons (spectators / adjacent courts) are excluded.
93- ""
94- # Which trajectory substrate seeds the windows (shuttle stays primary).
--
107- # < this. 0 = OFF.
108- min_players: int = Field(default=0, ge=0)
109- # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
110- court_polygon: Optional[list[list[float]]] = None
111- # Net line for the person two-sided-motion split (person features only ? the shuttle
112- # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
113- net_axis: Literal["x", "y"] = "y"
```

26. assistant (2026-06-19T17:00:16.257Z)

Confirmed: `court\_polygon` is a single static normalized polygon, and the design's only airborne-shuttle accommodation is the scalar `height\_pad` upward expansion in `court\_gate.py::admitted()`. There is no per-camera perspective height envelope, no descent/direction signal, and no player-anchor in the gate path today.

I have everything I need. The measured `\_patch\_key.json` data plus the overlay images directly confirm the perspective air-column overlap, and the code confirms the gate is a static floor polygon + uniform scalar upward pad. Let me synthesize the adversarial risks.

27. user (2026-06-19T17:01:16.175Z)

Structured output provided successfully

28. assistant (2026-06-19T17:01:31.649Z)

I've completed the adversarial review. Verdict: **design-needs-revision** (not killed ? the flaws are real but mitigable by changing the court-assignment mechanism).

The core finding, grounded in the project's own measured anchor data (`artifacts/multishuttle\_142/\_patch\_key.json`) and the overlay images (`tracks\_overlay\_crop.png`):

The design's softest spot is real and the shipped code (`backend/pipeline/detectors/court\_gate.py::admitted`) makes it worse than the doc admits. It disambiguates airborne shuttles by inflating a static floor polygon *upward* with a single scalar `height_pad`. But the two courts' air columns overlap in image-y exactly up high ? the measured data shows the **adjacent** shuttle sitting *higher* in frame (cy=171?207) than the **prominent** shuttle (cy=261) at the same moments, with 76% of window frames carrying both shuttles airborne. So any pad large enough to admit the prominent clear's apex (the track rose to y~259) necessarily sweeps across the adjacent court's air column and re-admits the very shuttle the gate exists to drop ? converting the claimed precision win into a precision leak. A single scalar pad is also perspective-wrong by construction (the doc's own §4.3 says it must vary across the frame, but the code takes one float), so it can pass the GX010142 litmus while overfitting and failing the multicourt-golden generalization.

The decisive mitigation across the high-severity risks is the same: **court assignment must be a track-level decision anchored to the low-y floor endpoints (serve origin / landing terminus) where the courts do NOT overlap ? not a per-point test at the high-y apex where they do.** This also unifies cleanly with the §4.5 shuttle-floor-contact rally-END primitive (the same descent-to-floor event both ends the rally and identifies its court) and with the route-2 player-foot anchor.

Two structurally important gaps I flagged: the C3 litmus (rally #1, single shuttle airborne) tests the *easy* case and never stresses the both-courts-airborne regime the recovery is born from; and the doc analyzes the prominent-clear-being-clipped failure but not the symmetric, data-dominant failure of an adjacent clear being admitted into the padded column.

Relevant files: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\court\_gate.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PROMINENT\_COURT\_DETECTION.md` (§4.3, §4.5, §5 experiment plan), `C:\Users\avidu\Projects\badminton-highlight-indexer\artifacts\multishuttle\_142\\_patch\_key.json`, and `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\fusion\_features.py` (`point\_in\_polygon`, `\_foot\_norm`).